Hello, Rails!

We can't help it—we just have to write a Hello, World! program to try a new system. Let's start by creating a simple application that sends our cheery greeting to a browser. After we get that working, we'll embellish it with the current time and links.

As you'll explore further in Chapter 3, *The Architecture of Rails Applications*, Rails is a model-view-controller (MVC) framework. Rails accepts incoming requests from a browser, decodes the request to find a controller, and calls an action method in that controller. The controller then invokes a particular view to display the results to the user. The good news is that Rails takes care of most of the internal plumbing that links all these actions. To write our Hello, World! application, we need code for a controller and a view, and we need a route to connect the two. We don't need code for a model, because we're not dealing with any data. Let's start with the controller.

In the same way that we used the `rails` command to create a new Rails application, we can also use a generator script to create a new controller for our project. This command is `rails generate`. So to create a controller called `say`, we make sure we're in the `demo` directory and run the command, passing in the name of the controller we want to create and the names of the actions we intend for this controller to support:

```
demo> bin/rails generate controller Say hello goodbye
      create  app/controllers/say_controller.rb
       route  get 'say/hello'
get 'say/goodbye'
      invoke  erb
      create    app/views/say
      create    app/views/say/hello.html.erb
      create    app/views/say/goodbye.html.erb
      invoke  test_unit
      create    test/controllers/say_controller_test.rb
      invoke  helper
      create    app/helpers/say_helper.rb
      invoke    test_unit
```